

1 Write a complete Python program for a simple Student Management System using classes and lists.

Aim

To write a Python program that manages student records (add, display, search, delete) using classes and lists.

Algorithm

1. Define a class with attributes: , , .
2. Define a class that:
 - Maintains a list of students.
 - Provides methods to add, display, search, and delete students.
3. Create a menu-driven program to interact with the system.
4. Perform operations based on user choice.

CODE

Student Management System using Classes and Lists

```
class Student:
```

```
    def __init__(self, roll_no, name, marks):
```

```
        self.roll_no = roll_no
```

```
        self.name = name
```

```
        self.marks = marks
```

```
    def display(self):
```

```
        print(f"Roll No: {self.roll_no}, Name: {self.name}, Marks: {self.marks}")
```

```
class StudentManagementSystem:
```

```
    def __init__(self):
```

```
self.students = []
```

```
def add_student(self, roll_no, name, marks):
```

```
    student = Student(roll_no, name, marks)
```

```
    self.students.append(student)
```

```
    print("Student added successfully!")
```

```
def display_all(self):
```

```
    if not self.students:
```

```
        print("No student records found.")
```

```
    else:
```

```
        print("\n--- Student Records ---")
```

```
        for student in self.students:
```

```
            student.display()
```

```
def search_student(self, roll_no):
```

```
    for student in self.students:
```

```
        if student.roll_no == roll_no:
```

```
            print("Student found:")
```

```
            student.display()
```

```
            return
```

```
    print("Student not found!")
```

```
def delete_student(self, roll_no):
```

```
    for student in self.students:
```

```
        if student.roll_no == roll_no:
```

```
            self.students.remove(student)
```

```
            print("Student deleted successfully!")
```

```
            return
```

```
print("Student not found!")
```

```
# Menu-driven program
```

```
def main():
```

```
    sms = StudentManagementSystem()
```

```
    while True:
```

```
        print("\n--- Student Management System ---")
```

```
        print("1. Add Student")
```

```
        print("2. Display All Students")
```

```
        print("3. Search Student")
```

```
        print("4. Delete Student")
```

```
        print("5. Exit")
```

```
    choice = input("Enter your choice: ")
```

```
    if choice == '1':
```

```
        roll_no = input("Enter Roll No: ")
```

```
        name = input("Enter Name: ")
```

```
        marks = input("Enter Marks: ")
```

```
        sms.add_student(roll_no, name, marks)
```

```
    elif choice == '2':
```

```
        sms.display_all()
```

```
    elif choice == '3':
```

```
        roll_no = input("Enter Roll No to search: ")
```

```
        sms.search_student(roll_no)
```

```
elif choice == '4':  
    roll_no = input("Enter Roll No to delete: ")  
    sms.delete_student(roll_no)  
  
elif choice == '5':  
    print("Exiting Student Management System. Goodbye!")  
    break  
  
else:  
    print("Invalid choice! Please try again.")  
  
if __name__ == "__main__":  
    main()
```

OUTPUT

--- Student Management System ---

1. Add Student
2. Display All Students
3. Search Student
4. Delete Student
5. Exit

Enter your choice: 1

Enter Roll No: 101

Enter Name: Yogin

Enter Marks: 89

Student added successfully!

Enter your choice: 2

--- Student Records ---

Roll No: 101, Name: Yogin, Marks: 89

Conclusion

- The program demonstrates a Student Management System using classes and lists.
- It supports adding, displaying, searching, and deleting student records.
- This is a foundation for building larger systems with features like updating records, storing in files/databases, and GUI/web integration

2 Write a program to search an element in a list using linear search.

Aim

To write a Python program that searches for a given element in a list using the linear search algorithm.

Algorithm

1. Define a function .
2. Traverse the list element by element.
3. Compare each element with the .
4. If found, return the index.
5. If not found after traversal, return .
6. Call the function with a sample list and display the result.

CODE

Program to search an element in a list using linear search

```
def linear_search(lst, key):  
    for i in range(len(lst)):
```

```

        if lst[i] == key:
            return i # Return index if found
        return -1 # Return -1 if not found

# Sample list
numbers = [12, 45, 78, 34, 89, 23]

# Input from user
key = int(input("Enter the element to search: "))

# Function call
result = linear_search(numbers, key)

if result != -1:
    print(f"Element {key} found at index {result}.")
else:
    print(f"Element {key} not found in the list.")

```

OUTPUT

```

Enter the element to search: 34
Element 34 found at index 3.

```

Conclusion

- The program demonstrates linear search, which checks each element one by one.
- It is simple but not efficient for very large lists compared to algorithms like binary search.
- Linear search is useful when the list is unsorted or small.